

Chapter 51: RxJS Internals

1. Overview

Every Angular application is, underneath, an RxJS application. `HttpClient`, the `Router`, reactive forms' `valueChanges`, `AsyncPipe`, `EventEmitter`, signals-to-observable interop — all of it sits on top of the same handful of classes: `Observable`, `Subscriber`, `Subscription`, and `operator/lift`. Volume 2 covered how to use RxJS operators productively. This chapter covers how RxJS is *built* — the actual object graph created when you call `.pipe().subscribe()`, why `Subscriber` is not just "the callbacks you passed in," how `Subscription` forms a teardown tree that unwinds deterministically, how schedulers turn synchronous code asynchronous by controlling *when* work executes, and how to write correct custom operators (including the classic leaked-inner-subscription bug that shows up in almost every `switchMap`-alternative interview question).

The goal is that after this chapter you can answer "what actually happens, object by object, when this code runs" for any RxJS snippet, and you can write a operator from scratch without copying an existing one.

2. Core Concepts

2.1 The four core classes and how they relate

RxJS's reactive core (`rxjs/src/internal`) is built from four classes that reference each other in a specific shape:

```
Observable ---subscribe()--> creates a Subscriber ---wraps--> Observer (your
next/error/complete)
    |
    | .source (for piped obs.)
    v
source Observable

    |
    | extends Subscription
    v
Subscription (teardown tree)
```

- `Observable<T>` — a *blueprint*. It stores a `_subscribe` function (or, for piped observables, an `operator` object plus a reference to the `source` Observable). Creating an Observable does nothing; only calling `.subscribe()` executes anything. This is the "unicast, lazy" contract.
- `Subscriber<T>` — the thing that actually receives `next / error / complete` calls and forwards them onward (to your callbacks, or to the next operator downstream). A `Subscriber` **is-a** `Subscription` (it extends it) **and** wraps an `Observer`. This dual nature is the single most important internals fact in RxJS: the object that delivers values is the *same* object that tracks teardown/unsubscription state.
- `Subscription` — a disposable handle. It tracks a `closed` boolean and a list of "teardown" logic (child subscriptions, unsubscribe functions, or other `unsubscribeables`) to run when `unsubscribe()` is called. It has no knowledge of values at all — it is purely resource-management plumbing.
- `operator` (legacy interface, mostly superseded by the `lift / operate` function pattern in modern RxJS) — an object with a `call(subscriber, source)` method that connects a downstream `Subscriber` to an upstream `Observable`.

2.2 Observable — what `subscribe()` actually does

Simplified from the real source (`Observable.prototype.subscribe`):

```

subscribe(observerOrNext?: Partial<Observer<T>> | ((value: T) => void)): Subscription {
  const subscriber = isSubscriber(observerOrNext)
    ? observerOrNext
    : new Subscriber(observerOrNext);

  // deprecated `operator` path (kept for understanding legacy `lift`)
  if (this.operator) {
    // operator.call wires subscriber -> source
    subscriber.add(this.operator.call(subscriber, this.source));
  } else {
    // no operator: this is a "leaf" Observable (of, interval, fromEvent, custom
    producer...)
    subscriber.add(
      this.source
        ? this._subscribe(subscriber)
        : this._trySubscribe(subscriber)
    );
  }

  return subscriber;
}

```

Key facts hiding in this snippet:

1. `subscribe()` returns the `subscriber`, not a separate object. The `Subscription` object you call `.unsubscribe()` on is the `Subscriber` that is receiving values. There is exactly one allocation doing double duty.
2. Every `.pipe(operatorFn)` call wraps the source in a *new* `Observable` whose `_subscribe` closes over the previous `Observable`. There is no operator chain stored as data structure to walk — it's a chain of *closures*, and it only gets invoked at `subscribe()` time, innermost-out.
3. **Nothing runs until `subscribe()` is called.** Building `a$.pipe(map(...), filter(...))` does zero work besides allocating a few `Observable` wrapper objects.

2.3 `pipe()` and `lift()` / `operate()`

`pipe()` itself is trivial — it's function composition:

```

pipe(...operations: OperatorFunction<any, any>[]): Observable<any> {
  return operations.length
    ? pipeFromArray(operations)(this)
    : this;
}

function pipeFromArray(fns) {
  return (source) => fns.reduce((prev, fn) => fn(prev), source);
}

```

So `source$.pipe(map(f), filter(g))` is literally `filter(g)(map(f)(source$))`. Each operator factory (`map`, `filter`, `switchMap`, ...) is a function that takes a source `Observable` and returns a **new** `Observable`. The interesting part is *how* that new `Observable` is built — this is what `lift()` (legacy) and `operate()` (modern, internal `rxjs/internal/util/lift`) do.

Legacy `lift()` pattern (still present for backward compatibility, and what most tutorials show):

```
// Observable.prototype.lift
lift<R>(operator: Operator<T, R>): Observable<R> {
  const observable = new Observable<R>();
  observable.source = this;
  observable.operator = operator;
  return observable;
}

// map operator, legacy style
function map(project) {
  return (source: Observable<T>) => source.lift(new MapOperator(project));
}

class MapOperator<T, R> implements Operator<T, R> {
  constructor(private project: (v: T, i: number) => R) {}
  call(subscriber: Subscriber<R>, source: Observable<T>) {
    return source.subscribe(new MapSubscriber(subscriber, this.project));
  }
}
```

Modern `operate()` pattern (used internally since RxJS 7, exposed conceptually via `new Observable(subscribe)` composition): most built-in operators are now written directly against `Observable`'s constructor plus a helper (`operate`) rather than the `Operator` class, because subclassing `Subscriber` per operator was expensive and made stack traces awful. The shape:

```
function map<T, R>(project: (value: T, index: number) => R): OperatorFunction<T, R> {
  return operate((source, subscriber) => {
    let index = 0;
    source.subscribe(
      createOperatorSubscriber(subscriber, (value) => {
        subscriber.next(project(value, index++));
      })
    );
  });
}

// operate() just builds `new Observable` and wires teardown/add so
// unsubscribing the *result* also unsubscribes the *source* subscription.
function operate(init) {
  return (source) => {
    if (hasLift(source)) {
      return source.lift(function (this: Subscriber<any>, liftedSource) {
        try {
          return init(liftedSource, this);
        }
      });
    }
  };
}
```

```

        } catch (err) {
            this.error(err);
        }
    });
}
throw new TypeError('Unable to lift unknown Observable type');
};
}

```

Both styles boil down to the same three responsibilities every operator implementation must fulfill:

1. Create a new `Observable` whose subscribe logic, when run, subscribes to the **source** with a **new Subscriber** (or "operator subscriber").
2. That new Subscriber's `next` / `error` / `complete` methods do the operator's transformation and then call the corresponding method on the **downstream** subscriber.
3. Whatever `Subscription` results from subscribing to the source must be **added as a child teardown** of the downstream subscriber, so that unsubscribing downstream also unsubscribes upstream.

2.4 The Subscriber's three-channel contract

An `Observer` is just an interface: `{ next(value), error(err), complete() }`. A `Subscriber` wraps an `Observer` and enforces the **Observable contract** on top of raw callbacks:

- **At most one of `error()` or `complete()` fires, ever, and it fires at most once.**
- **After `error()` or `complete()`, no further `next()` calls are honored** — the Subscriber marks itself `stopped` / `isStopped = true` and silently ignores anything after.
- **`next()` calls made after `unsubscribe()`** are also ignored — a closed Subscriber is inert.
- **Errors thrown inside a `next` handler are caught and routed to `error()`**, not left to bubble as an unhandled exception (this is what makes `subscribe({ next: fn })` "safe" — RxJS wraps your handler).

Simplified real behavior:

```

class Subscriber<T> extends Subscription implements Observer<T> {
    protected isStopped = false;

    next(value: T): void {
        if (!this.isStopped) {
            this._next(value);
        }
    }

    error(err: any): void {
        if (!this.isStopped) {
            this.isStopped = true;
            this._error(err);
        }
    }

    complete(): void {

```

```

    if (!this.isStopped) {
        this.isStopped = true;
        this._complete();
    }
}

unsubscribe(): void {
    if (!this.closed) {
        this.isStopped = true;
        super.unsubscribe(); // Subscription's teardown logic
    }
}
}

```

Note `unsubscribe()` also sets `isStopped = true` — a Subscriber that has been torn down for *any* reason (explicit unsubscribe, or upstream completed/errored) refuses to forward any further notifications. This is what makes "unsubscribe in `next()`" and "operator called `complete()` early" both safe: nothing downstream double-fires.

2.5 Subscription — the teardown tree

`Subscription` is deliberately dumb about values. Its entire job:

```

class Subscription implements Unsubscribable {
    public closed = false;
    private _teardowns: Set<Unsubscribable | (() => void)> | null = null;
    private _parentage: Subscription[] | Subscription | null = null;

    add(teardown: Unsubscribable | (() => void) | void): void {
        if (!teardown || teardown === this) return;
        if (this.closed) {
            // already closed -> execute teardown immediately, no need to store it
            execFinalizer(teardown);
        } else {
            if (teardown instanceof Subscription) {
                teardown._addParent(this);
            }
            (this._teardowns ??= new Set()).add(teardown);
        }
    }

    remove(teardown): void {
        this._teardowns?.delete(teardown);
    }

    unsubscribe(): void {
        if (this.closed) return;
        this.closed = true;

        // detach from parents so we don't get double-unsubscribed
        this._parentage && this._removeFromParents();
    }
}

```

```

    if (this._teardowns) {
      for (const t of this._teardowns) {
        execFinalizer(t); // calls t.unsubscribe() or t() as appropriate
      }
      this._teardowns = null;
    }
  }
}

```

This produces a **tree** (not a flat list): the outer `Subscription` returned by `.subscribe()` has children added via `subscriber.add(...)` for every intermediate operator's internal subscription to *its* source, all the way down to the origin producer's cleanup (e.g. `clearInterval`, DOM `removeEventListener`, `WebSocket` `close()`). Calling `.unsubscribe()` once at the top walks the whole tree depth-first and executes every leaf teardown — this is why unsubscribing a single top-level subscription in a component's `ngonDestroy` correctly tears down an entire `switchMap` / `combineLatest` / `shareReplay` pipeline underneath it.

`merge` d subscriptions, `takeuntil`, and manual `.add(fn)` calls all just add more nodes to this same tree.

2.6 Schedulers

A `Scheduler` answers one question: **when** does a scheduled unit of work actually execute? RxJS ships four production schedulers plus `Scheduler.now`:

Scheduler	Underlying primitive	"When"	Typical use
<code>queueScheduler</code>	Synchronous, recursive-call-safe queue	Immediately, but re-entrant calls are queued instead of nested (trampoline)	<code>scheduled(obs, queueScheduler)</code> , recursive schedule loops
<code>asapScheduler</code>	Microtask (<code>Promise.resolve().then()</code>)	End of current microtask queue, before macrotasks	Rare direct use; conceptually like <code>queueMicrotask</code>
<code>asyncScheduler</code>	<code>setInterval</code> / <code>setTimeout</code>	Macrotask, after a real (possibly 0ms) delay	<code>delay()</code> , <code>debounceTime()</code> , <code>throttleTime()</code> , <code>timeout()</code>
<code>animationFrameScheduler</code>	<code>requestAnimationFrame</code>	Before next repaint	animation-driven pipelines, <code>observeOn(animationFrameScheduler)</code> for UI batching

All of them implement the same interface: `schedule(work, delay?, state?)` returns a `Subscription`-like handle so scheduled work can be **cancelled** before it runs — this is critical: `debounceTime` cancels the previous scheduled emission by unsubscribing the previous scheduler action when a new value arrives, rather than letting a stale `setTimeout` fire.

Internally a `Scheduler` produces `Action` objects (`AsyncAction`, `AnimationFrameAction`, `QueueAction`) that:

1. Wrap the work function and its "this" `Scheduler`.
2. Register the real platform primitive (`setTimeout`, `requestAnimationFrame`, or push onto `queueScheduler`'s internal array).
3. Are themselves `Subscriptions` — `action.unsubscribe()` clears the pending timer/RAF/queue-entry.

`queueScheduler` deserves a special mention because it's the mechanism behind `Subject` / `BehaviorSubject` avoiding stack blowups on synchronous re-entrant emission and is also what `scheduled([1,2,3], queueScheduler)` uses to make even a plain array emit "asynchronously enough" to be lawfully composable, without an actual macrotask — everything still runs before the current call stack unwinds, just non-reentrantly

(a trampoline: if you're already inside a queue flush, new scheduled items are appended rather than recursively invoked).

`observeOn(scheduler)` and `subscribeOn(scheduler)` are the two operators that expose schedulers directly to users: `subscribeOn` controls **where the subscribe call itself executes** (useful to defer even the act of subscribing off the calling stack), `observeOn` controls **where each notification is delivered to the downstream subscriber** (re-emits every `next` / `error` / `complete` through `scheduler.schedule`).

2.7 How `switchMap`-family operators manage inner subscriptions

`switchMap`, `mergeMap`, `concatMap`, and `exhaustMap` all share the same skeleton — subscribe to an outer source, and for each outer value, subscribe to a derived "inner" Observable — but differ in **how many inner subscriptions can be active** and **what happens to a still-running inner subscription when a new outer value arrives**:

- `mergeMap`: unlimited concurrent inner subscriptions (or capped by a `concurrency` argument); never tears down a running inner early.
- `switchMap`: at most **one** inner subscription; a new outer value **unsubscribes** ("switches away from") the current inner before subscribing to the new one.
- `concatMap`: at most one inner subscription, but new outer values are **queued** until the current inner completes rather than cancelling it.
- `exhaustMap`: at most one inner subscription; new outer values arriving while an inner is active are **dropped** (ignored) instead of cancelled or queued.

The mechanism `switchMap` uses to cancel the previous inner is exactly `Subscription.add/remove`: it keeps a reference to the "inner subscriber" as an instance field, and on every new outer value it calls `this.innersubscription?.unsubscribe()` and `this.remove(this.innersubscription)` *before* creating and adding the new one. This is demonstrated concretely in the hand-rolled implementation below.

3. Code Examples

3.1 A minimal Observable implementation from scratch

This strips away every RxJS convenience (`lift`, error safety wrappers, static creators) to show the irreducible core: a class holding a subscribe function, and a `subscribe()` method that builds teardown-aware plumbing.

```
type NextFn<T> = (value: T) => void;
type ErrorFn = (err: any) => void;
type CompleteFn = () => void;

interface MiniObserver<T> {
  next: NextFn<T>;
  error: ErrorFn;
  complete: CompleteFn;
}

type TeardownLogic = (() => void) | void;

class MiniSubscription {
  private _closed = false;
```

```

private teardowns: Array<() => void> = [];

get closed() {
  return this._closed;
}

add(teardown: (() => void) | MiniSubscription | void): void {
  if (!teardown) return;
  if (this._closed) {
    // already closed: run immediately, don't leak
    typeof teardown === 'function' ? teardown() : teardown.unsubscribe();
    return;
  }
  this.teardowns.push(
    typeof teardown === 'function' ? teardown : () => teardown.unsubscribe()
  );
}

unsubscribe(): void {
  if (this._closed) return;
  this._closed = true;
  // Execute in LIFO order -- innermost-registered teardown first,
  // mirroring how nested resources should be released.
  while (this.teardowns.length) {
    const fn = this.teardowns.pop()!;
    try {
      fn();
    } catch (e) {
      // real RxJS aggregates these into an UnsubscriptionError; simplified here
      console.error('Error during teardown', e);
    }
  }
}
}

/**
 * MiniSubscriber: the dual-natured object.
 * - IS a MiniSubscription (extends it) -> can be unsubscribed, can hold child teardowns.
 * - WRAPS a MiniObserver -> enforces the next/error/complete contract.
 */
class MiniSubscriber<T> extends MiniSubscription implements MiniObserver<T> {
  private isStopped = false;

  constructor(private destination: Partial<MiniObserver<T>>) {
    super();
  }

  next(value: T): void {
    if (this.isStopped || this.closed) return;
    try {
      this.destination.next?.(value);
    } catch (err) {
      this.error(err);
    }
  }
}

```

```

    }
}

error(err: any): void {
    if (this.isStopped || this.closed) return;
    this.isStopped = true;
    try {
        this.destination.error?.(err);
    } finally {
        this.unsubscribe();
    }
}

complete(): void {
    if (this.isStopped || this.closed) return;
    this.isStopped = true;
    try {
        this.destination.complete?.();
    } finally {
        this.unsubscribe();
    }
}
}

class MiniObservable<T> {
    constructor(private _subscribe: (subscriber: MiniSubscriber<T>) => TeardownLogic) {}

    subscribe(observerOrNext: Partial<MiniObserver<T>> | NextFn<T>): MiniSubscription {
        const observer: Partial<MiniObserver<T>> =
            typeof observerOrNext === 'function' ? { next: observerOrNext } : observerOrNext;

        const subscriber = new MiniSubscriber<T>(observer);

        // Synchronously invoke the producer. Whatever it returns (a cleanup
        // function) is registered as a teardown on the very subscriber we hand back.
        let teardown: TeardownLogic;
        try {
            teardown = this._subscribe(subscriber);
        } catch (err) {
            subscriber.error(err);
            return subscriber;
        }
        subscriber.add(teardown);

        return subscriber;
    }

    pipe<R>(...ops: Array<(source: MiniObservable<any>) => MiniObservable<any>>):
    MiniObservable<R> {
        return ops.reduce((src, op) => op(src), this as MiniObservable<any>);
    }
}

```

```
// --- usage: a "producer" observable wrapping setInterval, with real teardown ---
const ticks$ = new MiniObservable<number>((subscriber) => {
  let count = 0;
  const id = setInterval(() => subscriber.next(count++), 1000);
  return () => clearInterval(id); // <- this is what MiniSubscription.add() stores
});

const sub = ticks$.subscribe({ next: (n) => console.log('tick', n) });
setTimeout(() => sub.unsubscribe(), 3500); // stops the interval after ~3 ticks
```

3.2 A custom operator from scratch: `retryWithBackoff`

A simplified but faithful reimplementation of retry-with-delay semantics, showing how an operator wires a new `observable` around a source, manages its own child subscription, and re-subscribes on error.

```
import { Observable, Subscriber, Subscription, MonoTypeOperatorFunction, timer } from
'rxjs';

/**
 * Re-subscribes to the source up to `maxRetries` times on error, waiting
 * `baseDelayMs * 2^attempt` between attempts (capped). Passes the error
 * through once retries are exhausted.
 */
export function retryWithBackoff<T>({
  maxRetries: number,
  baseDelayMs = 200,
  maxDelayMs = 5000
}): MonoTypeOperatorFunction<T> {
  return (source: Observable<T>) =>
    new Observable<T>((destination: Subscriber<T>) => {
      let attempt = 0;
      // holds whichever subscription is currently "live" -- either the
      // source subscription or a pending backoff-timer subscription.
      let current = new Subscription();

      const subscribeToSource = () => {
        const sourceSub = source.subscribe({
          next: (value) => destination.next(value),
          complete: () => destination.complete(),
          error: (err) => {
            if (attempt >= maxRetries) {
              destination.error(err);
              return;
            }
            const delayMs = Math.min(baseDelayMs * 2 ** attempt, maxDelayMs);
            attempt++;

            // Swap out the teardown target: drop the now-dead source
            // subscription and replace it with the pending timer so that
            // unsubscribing the OUTER observable during the backoff window
            // still cancels the retry.
            current = new Subscription();
```

```

        current.add(
            timer(delayMs).subscribe(() => {
                current.unsubscribe();
                subscribeToSource();
            })
        );
    },
    });
    current.add(sourceSub);
};

subscribeToSource();

// Teardown returned from the Observable constructor callback:
// called when `destination` (the downstream subscriber) unsubscribes,
// e.g. because the component was destroyed mid-retry-loop.
return () => current.unsubscribe();
});
}

// usage
// http.get('/flaky').pipe(retryWithBackoff(3, 250)).subscribe(...)

```

Two internals points this example exercises directly:

1. The **teardown** function returned from `new Observable(subscribeFn)` is exactly the mechanism **lift/operate** wire into `subscriber.add(...)` for built-in operators — the constructor form and the lift form are equivalent; `pipe()`-built operators just use `operate()` as sugar over this same pattern.
2. **current** is reassigned, not merely added to — this is the crucial "unsubscribe the stale resource before creating a new one" step. Forgetting to do this (only ever calling `.add()`, never swapping/unsubscribing the previous) is precisely the leak pattern covered in section 5.

3.3 Hand-rolled `switchMap` showing inner-subscription teardown

```

import { Observable, Subscription, ObservableInput, from, OperatorFunction } from 'rxjs';

export function mySwitchMap<T, R>(
    project: (value: T, index: number) => ObservableInput<R>
): OperatorFunction<T, R> {
    return (source: Observable<T>) =>
        new Observable<R>((destination) => {
            let outerIndex = 0;
            let innerSub: Subscription | null = null;
            let outerCompleted = false;
            let hasActiveInner = false;

            const checkComplete = () => {
                if (outerCompleted && !hasActiveInner) {
                    destination.complete();
                }
            };

            source.subscribe({
                next: (value) => {
                    if (innerSub) innerSub.unsubscribe();
                    innerSub = project(value, outerIndex).subscribe(destination);
                    hasActiveInner = true;
                },
                complete: () => {
                    outerCompleted = true;
                    checkComplete();
                }
            });
            outerIndex++;
        });
}

```

```

};

const outerSub = source.subscribe({
  next: (outerValue) => {
    // --- THE CORE OF SWITCH: cancel whatever inner is currently running ---
    if (innerSub) {
      innerSub.unsubscribe();
      innerSub = null;
      hasActiveInner = false;
    }

    let inner$: Observable<R>;
    try {
      inner$ = from(project(outerValue, outerIndex++));
    } catch (err) {
      destination.error(err);
      return;
    }

    hasActiveInner = true;
    innerSub = inner$.subscribe({
      next: (innerValue) => destination.next(innerValue),
      error: (err) => destination.error(err), // inner errors propagate immediately
      complete: () => {
        hasActiveInner = false;
        innerSub = null;
        checkComplete(); // outer may have already completed while this inner ran
      },
    });
  },
  error: (err) => destination.error(err),
  complete: () => {
    outerCompleted = true;
    checkComplete(); // only completes downstream if no inner is still running
  },
});

// Runs when the DOWNSTREAM subscriber unsubscribes (component destroyed,
// takeUntil fired, etc). Must tear down BOTH the outer and the currently
// running inner -- forgetting the inner is the classic leak (see Section 5).
return () => {
  outerSub.unsubscribe();
  innerSub?.unsubscribe();
};
});
}

```

Walk through what makes this correct, matched against real `switchMap` semantics:

- A new outer value **always** unsubscribes the previous inner (`innerSub.unsubscribe()`) before subscribing to the new one — this is literally "switch."
- Completion is deferred with a two-flag check (`outerCompleted && !hasActiveInner`) because the

outer can complete while an inner is still emitting — the operator must wait for the last inner to finish before completing downstream.

- The Observable-constructor teardown function unsubscribes **both** `outersub` and `innerSub` — an external unsubscribe (e.g., `ngonDestroy`) must cancel whatever inner HTTP call/timer is in flight, not just stop listening to the outer.

4. Internal Working

4.1 The Subscription tree end-to-end

Consider:

```
const sub = source$
  .pipe(
    switchMap((x) => inner$(x)),
    map((y) => y * 2)
  )
  .subscribe(consumer);
```

What object graph exists after this line runs (assuming everything is synchronous long enough to produce a first inner subscription)?

```
sub (= Subscriber wrapping `consumer`, created by map's operator; returned to caller)
└─ child: switchMap's internal subscriber (subscribed to the switchMap-lifted Observable)
    └─ child: outersub (subscription to source$)
        └─ child: innerSub (subscription to the CURRENT inner Observable, replaced on each
            switch)
```

Calling `sub.unsubscribe()` triggers, top-down: `map`'s Subscriber closes → its child (switchMap's Subscriber) closes → *its* children (`outersub`, `innerSub`) close, each running their own producer teardown (e.g. an HTTP subscription's abort, or `clearInterval`). This is why a **single** `.unsubscribe()` call or a **single** `takeUntil(destroy$)` at the top of a long `pipe()` chain is sufficient to release everything beneath it — the tree, not a flat list, guarantees full-depth cleanup.

Critically, each operator is responsible for calling `.add()` (or returning a teardown function, same thing) on its **own** subscriptions to sources/inneres; RxJS does not automatically discover "orphan" subscriptions. A hand-written operator that subscribes to an inner Observable but never adds that subscription as a child of the outer chain will not be torn down by an upstream unsubscribe — the inner keeps running.

4.2 `lift()/operate()`: building wrapper Observables

The mental model that resolves most confusion: `.pipe(op)` **never mutates the source Observable**. It always allocates a *new* Observable object. That new Observable's `_subscribe` function, when eventually invoked (only at terminal `.subscribe()` time), does three things in order:

1. Creates an "operator Subscriber" whose `next / error / complete` implement the operator's logic and forward to the **downstream** subscriber it was constructed with.
2. Calls `source.subscribe(operatorSubscriber)` — subscribing to the *original* upstream Observable.
3. Registers the `Subscription` that call returns as a child teardown of the downstream subscriber

(`destination.add(...)` or the `operate()` /constructor-return equivalent).

Because each `.pipe(...)` call nests one more layer of "new Observable wrapping old Observable," subscribing to the outermost result triggers a cascade of nested `subscribe()` calls that unwinds from the **last** operator to the **first**, and — because subscribing to a plain synchronous producer (`of(1,2,3)`, a synchronous `new Observable` callback) emits immediately inside that call — values actually flow from the **first** operator to the **last** as soon as the *innermost* subscribe reaches its producer code. Construction order (outer→inner) and data-flow order (inner→outer) are opposite, which is a frequent source of confusion when reading stack traces from thrown errors inside operators (the stack shows nested `next` calls from source producer outward through every operator's Subscriber to your final callback).

4.3 Scheduler internals: microtask vs macrotask vs animation frame

- `queueScheduler` never touches the browser event loop at all in the common case — its `flush()` uses a simple array-based trampoline. If you're not already inside a flush, scheduling starts one immediately (still synchronous, same tick). If you *are* already inside a flush (re-entrant scheduling, e.g. a `subject.next()` triggering another `subject.next()` synchronously), the new action is appended to the array instead of being invoked via a nested function call — this is what prevents `RangeError: Maximum call stack size exceeded` on deeply re-entrant synchronous chains, at the cost of ordering: reentrant work runs *after* the current action finishes, not nested inside it (breadth-first instead of depth-first).
- `asapScheduler` schedules via a microtask (`Promise.resolve().then(flush)` internally, guarded by the same trampoline as `queueScheduler`). Microtasks always drain completely before the browser proceeds to the next macrotask or repaint — so `asapScheduler` work always runs before any pending `setTimeout(0)` and before the next paint, but after all sitting synchronous code finishes.
- `asyncScheduler` schedules via `setTimeout / setInterval` (real macrotask). Even `asyncScheduler.schedule(fn, 0)` yields to the event loop — pending UI events, other macrotasks, and a repaint can all happen before it fires. This is why `debounceTime(0)` is not equivalent to synchronous execution and can reorder relative to microtask-based work.
- `animationFrameScheduler` schedules via `requestAnimationFrame`, meaning the callback runs once per display refresh, immediately before the browser's next paint/layout pass — useful for coalescing many rapid emissions (e.g. drag events) down to "once per visual frame" rather than once per microtask/macrotask, which would still be far more often than the screen can show.

The ordering guarantee interviewees are expected to know cold: **synchronous code** → **microtasks** (**Promises**, `asapScheduler`, `queueMicrotask`) → **macrotasks** (`setTimeout`, `setInterval`, `asyncScheduler`) → `requestAnimationFrame / animationFrameScheduler` → **repaint**, repeating every event-loop turn.

4.4 `subscribeOn` vs `observeOn` internals

- `subscribeOn(scheduler)` wraps the **act of calling** `source.subscribe(...)` in `scheduler.schedule(() => source.subscribe(subscriber))`. It affects only *when subscription itself begins* — relevant because some producers do synchronous work immediately inside their subscribe function (e.g., synchronously reading a DOM property, or a `Subject` that replays nothing but subscribes eagerly). Placing it anywhere in the pipe affects the whole chain's subscribe timing (it's an "upstream-in-time" effect, not positional in the operator sense) since only one subscribe call ever actually reaches the raw producer.

- `observeOn(scheduler)` instead wraps **each notification delivered to its downstream Subscriber**: every `next / error / complete` call it receives is re-issued via `scheduler.schedule(() => destination.next(value))` rather than being forwarded synchronously. Unlike `subscribeOn`, `observeOn`'s effect is positional — only notifications after that point in the pipe are rescheduled.

5. Edge Cases & Gotchas

1. Leaking a stale inner subscription in a hand-rolled `switchMap`-like operator.

If your custom operator subscribes to an inner Observable but forgets to unsubscribe the *previous* inner before subscribing to a new one (i.e., only ever does `innerSub = inner$.subscribe(...)` without first calling the old `innerSub?.unsubscribe()`), you silently get `mergeMap` semantics while believing you wrote `switchMap` — every previous inner (e.g., an in-flight HTTP request, an open WebSocket, a running `setInterval`) keeps emitting and keeps its resources alive indefinitely. This is invisible in simple tests (values still arrive, tests still pass) and only surfaces as memory growth / duplicate network calls / race-condition bugs under real, fast-firing input (type-ahead search is the textbook victim).

2. Forgetting to add the inner subscription as a teardown of the outer chain.

Even with correct switch-away logic, if the *currently active* inner subscription is not registered so that an external `unsubscribe()` (component destroy, `takeUntil`) tears it down, destroying the component leaves one last in-flight inner request/timer running forever — a leak that only manifests on the "last" value before teardown, making it easy to miss in manual testing but guaranteed in production traffic.

3. Synchronous vs asynchronous subscribe timing surprises.

`new Observable(subscriber => { subscriber.next(1); subscriber.complete(); })` delivers `1` and completes **synchronously, inside the `.subscribe()` call**, before `.subscribe()` even returns a `Subscription` to the caller. Code that does:

```
let sub: Subscription;
sub = source$.subscribe(v => { if (v === 1) sub.unsubscribe(); });
```

will throw `ReferenceError / TypeError` (using `sub` before assignment) for a *synchronous* source, but work fine for an asynchronous one — a classic RxJS gotcha caused by assuming all sources behave like a `Promise` (always at least one microtask away). `Subject`, `of()`, `EMPTY`, and any custom synchronous producer all emit before `subscribe()` returns.

4. `BehaviorSubject / ReplaySubject` re-entrancy.

Because `Subject.next()` iterates its observer list synchronously, a `next()` call from *inside* a subscriber's `next` callback (re-entrant emission) can interleave in surprising order relative to late subscribers added mid-emission — RxJS snapshots the observer array at the start of the emit loop in modern versions specifically to avoid "subscriber added during emit also receives this same emit" bugs, but older custom Subject-like implementations that iterate a live array are vulnerable to this.

5. Custom operators that don't propagate errors from the inner Observable.

It's common to write a custom operator's inner subscription handler with only `next / complete` and no `error`, assuming errors "bubble up" automatically. They do not — RxJS's Subscriber contract requires **you** to explicitly call `destination.error(err)` in your inner error handler; an inner Observable erroring silently (no `error:` handler wired) results in the inner simply stopping — the outer pipeline appears to hang, never completing and never erroring, because nothing ever calls `destination.complete()` or `destination.error()`.

6. Executing side effects in `map`'s projection function relying on "runs once per subscriber."

Because Observables are unicast and lazy by default, a `source$.pipe(map(sideEffectfulFn))` re-executes `sideEffectfulFn` **independently for every subscriber**, including for accidental multiple subscriptions (e.g., binding the same observable with `| async` in two places in a template without `shareReplay`). This isn't a bug in `map`; it's a direct consequence of the "cold, no-shared-execution" internals — every `subscribe()` call replays the entire operator chain and re-invokes the source producer from scratch.

7. `queueScheduler` reentrancy changes emission order, not timing, and this is easy to misdiagnose as an "async" issue.

Because `queueScheduler`'s trampoline defers reentrant scheduled work to *after* the current action rather than nesting it, code assuming strict call-order (`a` before `b` because `a` was scheduled first inside `b`'s execution) can observe values delivered breadth-first rather than depth-first — this is still perfectly synchronous (same tick, before `subscribe()`'s caller resumes only if nothing else scheduled it later), so logging "async" is the wrong diagnosis; the real cause is trampoline reordering.

8. Forgetting that `finalize()` runs on unsubscribe too, not just on error / complete.

A custom operator (or a `finalize()` usage) that assumes its cleanup callback only fires on stream termination will be surprised that it *also* fires on manual `unsubscribe()` before completion — which is correct/intended RxJS behavior (teardown must run in all three cases: error, complete, unsubscribe) but trips up authors who write "cleanup" logic assuming a terminal notification necessarily occurred first.

6. Interview Questions & Answers

Q1. What is the actual relationship between `Subscriber` and `Subscription`?

A: `Subscriber` extends `Subscription` and also implements the `Observer` interface (`next` / `error` / `complete`). It is the single object that both receives values from a producer/operator and can be unsubscribed to stop that flow and run cleanup. There is no separate "value-receiving object" and "cancellation-handle object" — they're the same instance, which is why `source.subscribe(...)` returns something you can both observe was created from your callbacks and call `.unsubscribe()` on.

Q2. Does creating an Observable with `.pipe()` do any work?

A: No. `.pipe(op1, op2)` only allocates new `Observable` wrapper objects whose subscribe logic closes over the previous Observable/operator. No producer runs, no operator logic executes, until a terminal `.subscribe()` call reaches all the way down to the innermost source's subscribe function. This laziness is why the same piped Observable can be subscribed to multiple times and re-execute its entire chain (including side effects) each time.

Interviewer intent: checks whether the candidate distinguishes "declarative pipeline construction" from "execution," a distinction that trips up developers coming from Promise-based mental models where `.then()` chains start executing as soon as constructed.

Q3. What does `subscribe()` return, and what is it, precisely?

A: It returns a `Subscription` — in practice, the `Subscriber` instance created (or passed in) for that call. Calling `.unsubscribe()` on it walks a tree of child teardowns registered via `.add()` during the subscribe cascade, releasing every resource acquired by every operator and the origin producer in the pipeline.

Q4. Why can a `next()` call after `error()` or `complete()` never reach your subscribe callback?

A: `Subscriber` tracks an internal `isStopped` flag, set to `true` the moment `error()` or `complete()` is invoked (before forwarding to your Observer). Every subsequent `next()` / `error()` / `complete()` call checks this flag first and is a no-op if already stopped. This enforces the Observable contract (at most one terminal

notification, no notifications after it) regardless of whether the *source* itself misbehaves and calls `next` after `complete` — the contract is enforced at the Subscriber layer, not trusted to producers.

Interviewer intent: tests whether the candidate knows the contract is defensively enforced in the Subscriber itself, not merely a convention producers are expected to follow — i.e., RxJS protects consumers even from buggy custom Observables.

Q5. Explain what `lift()` does and why modern RxJS moved away from it for built-in operators.

A: `lift(operator)` creates a new `Observable` that stores a reference to the current one as `.source` and the given `operator` object (with a `call(subscriber, source)` method) as `.operator`. When subscribed, `Observable.prototype.subscribe` sees `this.operator` is set and calls `operator.call(newSubscriber, this.source)`, which typically subscribes to `source` with a specialized `Subscriber` subclass implementing the operator's logic. Modern RxJS (v7+) largely replaced per-operator `Subscriber` subclasses with a single generic `operate()` helper plus plain closures (`createOperatorSubscriber`), because allocating and subclassing a `Subscriber` per operator per subscription was measurably slower and produced deep, hard-to-read call stacks; the closure-based approach achieves the same wiring with less overhead while keeping `lift`'s public surface (`.lift()` is still callable) for library-author backward compatibility.

Q6. Walk through, object by object, what happens when you call `.unsubscribe()` on the result of `a$.pipe(switchMap(...), map(...)).subscribe(fn)`.

A: The call returns the outermost `Subscriber` (built for `map`'s operator, wrapping your `fn`). Unsubscribing it: sets its own `closed = true`, then iterates its registered teardowns — one of which is the `Subscriber` created by `switchMap`'s operator for subscribing to the `switchMap`-lifted `Observable`. That inner `Subscriber`'s `unsubscribe()` in turn runs its own teardowns: the subscription to the outer source `a$`, and — if one is currently active — the subscription to the current inner `Observable`. Each of those, if wrapping a real producer (HTTP call, timer, DOM listener), executes its actual cleanup (abort, `clearInterval`, `removeEventListener`) at the leaf. The whole cascade is synchronous and depth-first.

Q7. What's the difference, mechanically, between `subscribeOn` and `observeOn`?

A: `subscribeOn(scheduler)` schedules the *single* call to `source.subscribe(...)` through the given scheduler — it affects when the producer's setup code runs, and because subscribing only happens once per chain regardless of operator count, its position in `.pipe()` doesn't change which producer code gets deferred (only one raw `subscribe` call exists). `observeOn(scheduler)` instead intercepts every `next` / `error` / `complete` notification *at that point in the pipe* and re-emits each one via `scheduler.schedule(...)`, so it is positional — operators before it in the pipe run synchronously as before, while everything downstream of it receives notifications only after the scheduler releases them.

Q8. Why does `debounceTime(0)` still change behavior compared to no debounce at all, if both look "immediate"?

A: `debounceTime` uses `asyncScheduler` internally (macrotask, `setTimeout`), even with a delay of `0`. A `setTimeout(fn, 0)` still yields to the event loop — it runs after all currently queued synchronous code and all pending microtasks, and after any pending UI events already queued. So `debounceTime(0)` reorders relative to microtask-based work (Promises) and to other operators using `asapScheduler` / `queueScheduler`, even though "0ms" sounds instantaneous.

Interviewer intent: probes whether the candidate actually understands the micro/macrotask distinction rather than treating "delay: 0" as "synchronous."

Q9. How would you implement a minimal custom operator that behaves like `retry(n)`? What's the key mechanism?

A: Wrap the source in `new Observable(destination => { ... })`. On each attempt, subscribe to the source, forwarding `next` / `complete` straight to `destination`, but on `error`, if attempts remain, re-subscribe to the source instead of forwarding the error (incrementing an attempt counter), otherwise call `destination.error(err)`. The key mechanism is **subscription replacement**: each retry must discard the old (errored) source subscription and create a fresh one, and the *current* subscription must be tracked in a mutable variable so that the teardown function returned from the Observable constructor can always unsubscribe whichever subscription is currently live — including mid-backoff-delay if using a scheduler-based wait between retries.

Q10. What causes a memory/resource leak in a hand-written `switchMap`, and how do you avoid it?

A: Two independent mistakes cause it: (1) not unsubscribing the *previous* inner subscription before creating a new one when a new outer value arrives — this silently degrades into `mergeMap` behavior, letting every previous inner keep running; (2) not including the *currently active* inner subscription in the teardown function returned to the Observable constructor (or not adding it as a child of the outer Subscription) — this means an external `unsubscribe()` (e.g., component destroy) stops listening to new outer values but leaves the last in-flight inner (HTTP call, timer, socket) running forever. The fix is to keep a single mutable reference to the current inner `Subscription`, explicitly `.unsubscribe()` it both on new-outer-value and in the overall teardown function, and only ever have at most one live inner subscription tracked at a time.

Q11. Why does a Subject re-entrant `next()` call (calling `.next()` again from inside a subscriber's `next` handler) not stack-overflow, and what determines the order of delivery?

A: `Subject` maintains an array/list of active observers and iterates it synchronously per `next()` call using a straightforward loop (not literal recursion into the scheduler's trampoline unless a scheduler-backed Subject/operator is involved) — a reentrant `next()` call just starts a new synchronous iteration over the (typically snapshotted) observer list, which unwinds and returns before the outer iteration's loop continues to its next observer, so it's ordinary nested function calls, bounded by the number of active subscribers, not unbounded recursion. If a scheduler like `queueScheduler` is involved instead (e.g. via `scheduled()`), *that* is where the trampoline specifically prevents stack growth by queueing reentrant scheduled actions rather than nesting them — but a plain `Subject.next()` reentrant call is just an inner synchronous call stack, and delivery order is depth-first: the reentrant `next()`'s observers fully finish before control returns to continue notifying the remaining observers of the outer call.

Interviewer intent: separates candidates who've memorized "RxJS uses a trampoline to avoid stack overflow" as a blanket fact from those who know it's specifically a scheduler-queue mechanism, not a property of every synchronous emission path (a plain `Subject` fanning out to a huge number of active subscribers reentrantly *can* still deepen the call stack, it's just bounded by subscriber count rather than by emitted-value count).

Q12. If you call `.subscribe()` twice on the same piped Observable, do both subscriptions share the same operator instances/state?

A: No — for a plain (non-multicasted) Observable, each `.subscribe()` call re-runs the entire wrapping chain from the outside in, creating brand-new operator Subscribers and a brand-new subscription to the original source producer for that call only. Any internal state an operator keeps (e.g., `scan`'s accumulator, `switchMap`'s "current inner" reference) is scoped to that one subscription's closure, not shared across subscribers. Sharing execution/state across multiple subscribers requires an explicit multicasting operator (`share`, `shareReplay`, `connect` / Subject-based composition) — a Volume 2 topic, but its internals-relevant consequence is: multicasting works by inserting a `Subject` as the single actual subscriber to the source, and each "outer" subscriber merely subscribes to that `Subject` instead of re-triggering the source chain.

Q13. What determines whether errors thrown inside your `next` callback passed to `.subscribe()` become an unhandled exception or route through RxJS's error channel?

A: `Subscriber.next()` invokes your `next` handler inside a `try/catch` (this wrapping is done when the Subscriber is constructed from a plain function/partial-observer, via `createOperatorSubscriber`-style safe wrapping in modern RxJS, or the older `SafeSubscriber` in legacy versions). Any exception thrown synchronously inside your `next` callback is caught and redirected into `subscriber.error(err)`, which then stops further `next` calls and delivers the exception to your `error` handler if one was supplied (or, absent one, RxJS reports it as an "unhandled error," historically by rethrowing asynchronously). This is why a bug in a `.subscribe(v => risky(v))` call doesn't crash the surrounding synchronous call stack — RxJS treats it as a stream failure, not a raw exception.

Q14. Why must a custom operator use `operate()` /the Observable constructor rather than mutating the source Observable in place?

A: Observables must remain reusable/composable — a single `source$` may be subscribed to many times, potentially by different consumers applying different downstream operators. If an operator mutated `source$` directly (rather than producing a new wrapper Observable), every other consumer of that same reference would unexpectedly pick up the operator's behavior, and repeated `.pipe()` calls on the same base Observable would corrupt each other. Returning a new Observable per operator application preserves referential purity: `source$.pipe(op)` never affects `source$` itself, so `source$` can still be subscribed to elsewhere, unmodified, and multiple different `.pipe()` chains can be built from the same source independently.

7. Quick Revision Cheat Sheet

- `observable` = lazy blueprint holding `_subscribe` (or `source` + `operator`). No work happens until `.subscribe()`.
- `Subscriber` = `Subscription` (cancellation) + `Observer` (next/error/complete) in one object; enforces "at most one terminal event, nothing after unsubscribe" via `isStopped` / `closed`.
- `Subscription` = pure resource-management tree; `.add()` registers child teardowns (functions or other Subscriptions); `.unsubscribe()` walks and executes them all, once, depth-first.
- `pipe(op1, op2) = op2(op1(source))`; each operator factory returns a *new* wrapping Observable — never mutates the source.
- `lift()` / `operate()` = the mechanism each operator uses to: (1) build a new Observable, (2) subscribe to the source with a specialized Subscriber, (3) register that subscription as a teardown child of the downstream Subscriber.
- **Errors inside `next` callbacks** are caught and routed to `error()` automatically — you don't get raw unhandled exceptions from `.subscribe(fn)`.
- **Schedulers control *when*, not *what***: `queueScheduler` = sync trampoline (reentrancy-safe, same tick); `asapScheduler` = microtask; `asyncScheduler` = macrotask (`setTimeout`); `animationFrameScheduler` = before next repaint (`requestAnimationFrame`).
- **Ordering guarantee**: sync code → microtasks → macrotasks → animation frame → repaint, every event-loop turn.
- `subscribeOn` = defers *where the subscribe call happens* (affects the one real producer subscribe).
`observeOn` = defers *delivery of each notification* from that point in the pipe onward (positional).
- `switchMap` cancels the previous inner before subscribing to a new one; `mergeMap` never cancels;

`concatMap` queues instead of cancelling; `exhaustMap` drops new outer values while an inner is active.

- **The #1 custom-operator bug:** not unsubscribing the *previous* inner subscription (leak → accidental `mergeMap` semantics) and/or not including the *current* inner in the operator's own teardown function (leak survives even after the consumer unsubscribes).
- **Cold/unicast by default:** re-subscribing to the same piped Observable re-runs the whole chain and the source producer from scratch, with independent operator state per subscription; multicasting (`share` , `subject`) is required to share execution.

Created By - Durgesh Singh